PROJECT-BASED LEARNING

PHASE-II EVALUATION

Food Surplus Management and Redistribution System

Team ID: DSCPP-III-2026-T074

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T074
Semester: 3rd
Section: ML-1, ML-2, ML- 4, DS-1
Domain: C++ Object Oriented Programming and Data Structures

Team Members

1. Nidhyati Bisht – 2512510007
2. Subhansh S. Rawat – 2510370047
3. Jatin S. Mehta – 2510370090
4. Vartika Gosai – 2510370485

Mentor:

Mr. Kartikey Arora

Interactions completed: 8

Phase-I Recap & Feedback Incorporated

Phase-I Status

Briefly During Phase-I, we proposed and built the foundational architecture of the Food Surplus Management and Redistribution System, connecting donors with recipient NGOs using a C++ backend and a web interface. We successfully implemented object-oriented programming and data structures, including min-heap priority queues for expiry tracking, FIFO queues for request management, and hash tables for allergen lookups.

Feedback Received

- *Feedback 1- Replace text files with a structured database for reliable storage.*
- *Feedback 2- Build a server connection layer to link the frontend with the backend*
- *Feedback 3- Host the project on a local web server for smooth live demonstrations.*

Changes Incorporated

- *Modification 1: Integrated SQLite database for proper data storage.*
- *Modification 2: Created a Python server to link the website with backend.*
- *Modification 3: Deployed a local server on port 8080 for live browser demonstrations.*

Problem Statement & Finalized Objectives

Problem Statement

Every day, large quantities of usable food are wasted across canteens, events, and messes while a significant portion of society faces food insecurity. Furthermore, reliance on manual tracking and coordination leads to distribution delays, missed expiration windows, and inefficient matching with recipient NGOs.

Finalized Objectives

1. *Integrate the web frontend with the Python Flask API and SQLite database.*
2. *Automate user registration and persistent relational data storage.*
3. *Execute C++ core backend logic for priority-based food matching and queueing.*
4. *Host a local server environment on port 8080 for live operational demonstrations.*
5. *Implement inventory status tracking and request verification workflows.*

Updated System Architecture / Workflow



Major Components

- *HTML, CSS, JS User Interface for donor and recipient*
- *Python Flask Server for routing.*
- *Compiled C++ engine for OOP and data structures.*
- *SQLite Database for persistent storage.*

Changes Since Phase-I

- *Architecture change: Static file handling to client-server setup.*
- *New component: Python Flask API and SQLite database added.*
- *Improved workflow: Enabled live local server operations on port 8080 for real-time execution.*

Module Design & Functional Requirements

Major Modules

1. *Donor Module*
2. *Recipient Module*
3. *Matching Module*
4. *Verification Module*

Functional Requirements

- *Donors can securely submit food details.*
- *Recipients can view live listings and submit food requests.*
- *Execute priority based food matching.*
- *Update pickup status and verification.*

Module Dependency

The frontend web interface depends on the Python Flask API to handle user requests. The Flask API interacts with the SQLite database for permanent data storage and triggers the compiled C++ backend engine to run matching logic, finally returning the results back to the user interface.

Technology Stack & Development Methodology

Technology Stack

- Programming: *C++, Python*
- Database: *SQLite*
- Framework: *Flask (Python)*
- Development Tool: *VS Code and Local Server (Port 8080)*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis: Defining user roles and core features.
2. System Design: Planning the 3-tier architecture and database.
3. Module Development: Coding UI, Flask API, C++ backend, SQLite.
4. Integration: Connecting frontend, Flask, database, and C++ engine.
5. Testing & Validation-Running local server and verifying workflows.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Linked Lists, FIFO Queue, Min. Heap, Hash Table*	*Matching Module*
OOPs with C++	*Class and Objects, Inheritance, Polymorphism, STL, Encapsulation*	*Users Module*
Operating Systems	*No specific OS concept used in this project phase.*	*None*
DBMS	*SQL, Queries, Relational Tables, SQLite Storage*	*Database/ Verification module*

Development & Implementation Progress

Module	Status	Work Completed
Donor Module	Completed	*User registration, login, and food surplus posting form UI with Flask routing.*
Recipient Module	Completed	*NGO registration, live food listings view, and request submission workflow.*
Matching Module	Completed	*C++ backend integration for priority-based sorting, allergen lookup and expiry management.*
Verification Module	In Progress	*Database schema and basic status routing set up; dual-sided confirmation workflow refinement ongoing.*

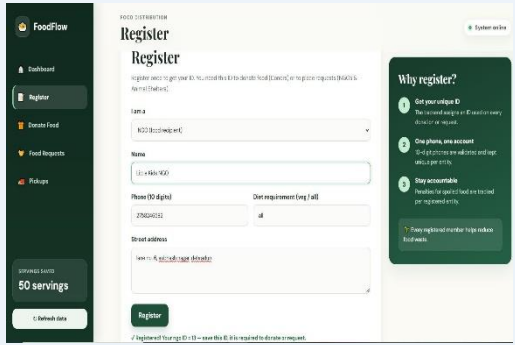
Overall Project Completion: *85%*

Working Prototype / Demonstration

Implemented Features

- *Web forms on port 8080 for donations and claims.*
- *C++ Min-Heap prioritizing expiry meals*
- *Hash Table blocking allergens and animal toxins.*
- *Two-way pickup (Donor/Recipient).*
- *SQLite database for persistent data storage.*

Prototype / Output



Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Expiry Heap	Shortest shelf-life out first.	Priority batch surfaced.	Pass
Allergen Gate	*Block unsafe meals*	Allergen matches rejected	Pass
Phone Validation	*10 digits & unique only*	Rejects invalid/duplicates	Pass

Results /Validation

Live server tests on port 8080 show proper accuracy in expiry sorting, allergen gating, and SQLite foreign key checks. The users are being registered properly and being assigned IDs after registration. Food requests and donations are being registered according to Recipient/Donor ID.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: https://github.com/nidhyatibisht-2026/DSCPP-III-2026-T074_PBL_SEM3
- README.md: Updated
- Contributors: All Members Added
- Repository Status: Updated
- Branches: None except main branch

Development Activity

- *Regular commits maintained*
- *Meaningful commit messages*
- *Individual contribution visible*
- *Latest implementation uploaded*
- *Documentation maintained*

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Nidhyati Bisht	Team Lead / DBMS, Frontend and Backend Integration	*Designed SQLite relational schema, wrote Python database test scripts, and linked Flask routes.*	30
Subhansh S. Rawat	Backend developer, testing and debugging	*C++ OOP classes, Min-Heap priority queue, boundary test suite*	13
Jatin S. Mehta	Frontend developer	*HTML/CSS layout, client-side JS form validation.*	10
Vartika Gosai	Backend developer	*C++ FIFO request queue, Hash Table allergen lookup*	9

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Validating Min-Heap urgency without real-time feeds.*
- *Connecting Flask API cleanly with compiled C++.*
- *Input stream errors on invalid/duplicate phone numbers.*

Solutions / Learning

- *Verified Min-Heap priority sorting using synthetic timestamps.*
- *Unified schema between forms, Flask API, and C++ engine.*

Phase-III Roadmap

1. *Finalize two way confirmation in SQLite database.*
2. *Run full multi-user trials on local network.*
3. *Allow admin access to user and food data through frontend.*
4. *Package final code and complete documentation.*

Expected Outcomes & References

Expected Final Outcomes

- *Complete 3-tier web platform for surplus food rescue.*
- *Instant automated matching with zero allergen errors.*
- *Proper storage and retrieval of food and user data as per requirement.*

References

1. *Cattaneo et al. (2021). Reducing Food Loss and Waste, Food Policy.*
2. *Patel, R., et al., "Smart Food Distribution System," J. Tech., vol. 15, pp. 325–342, 2023.*
3. *Kamran, R., & Sundarakani, B., "Food Safety Assurance," IEEE ICTMOD, 2024.*
4. *Stroustrup, B., The C++ Programming Language, 4th ed., Addison-Wesley, 2013.*
5. *GitHub Open Source, Food Waste Management Repo, GitHub.*

Thank You
Questions & Suggestions